# Combining plots in ggplot2

Pawan · Follow
8 min read · Dec 22, 2022



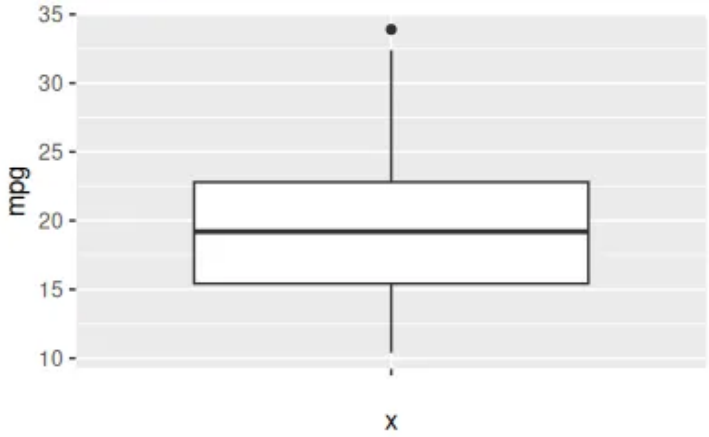
The ggplot2 package doesn't provide a function to arrange multiple plots in a single figure. Still, some packages allow combining multiple plots into a single figure with custom layouts, width, and height, such as **cowplot**, **gridExtra**, and **patchwork.**

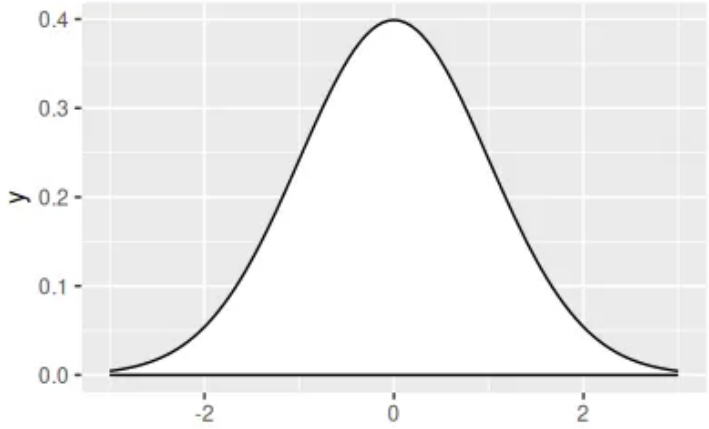
### Sample plots

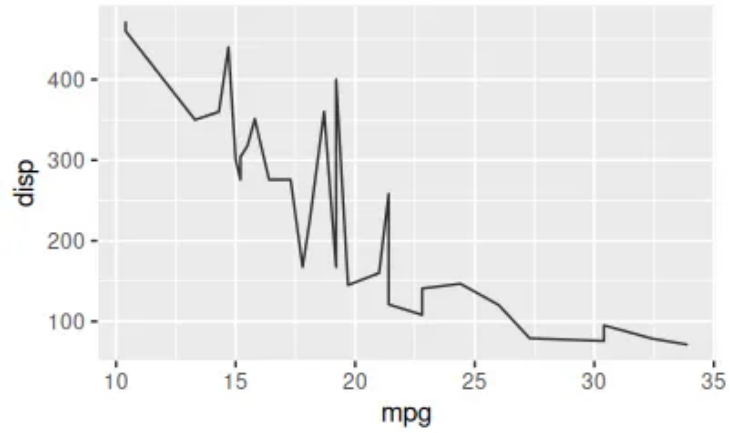
Let's create four different plots and assign them to objects named **p1, p2, p3,** and **p4.** You can use any other names.

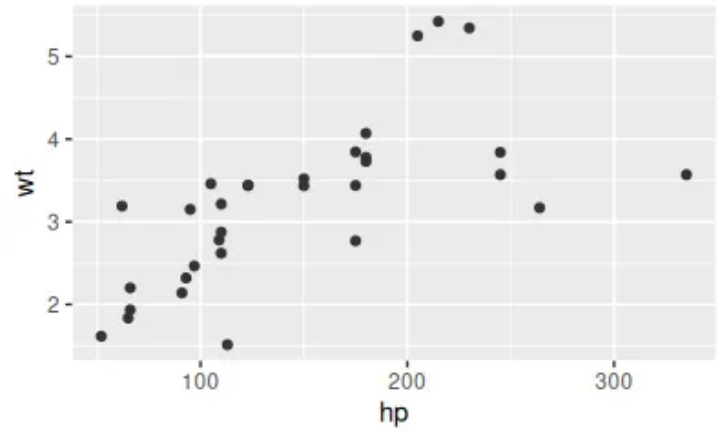
```
library(ggplot2)

#Data
mtcars->df

# Box plot
p1 <- ggplot(df, aes(x = "", y = mpg)) +
  geom_boxplot()

# Density plot
p2 <- ggplot() +
  stat_function(fun = dnorm, geom = "density",
                xlim = c(-3, 3), fill = "white")

# Line chart
p3 <- ggplot(df, aes(x = mpg, y = disp)) +
  geom_line(color = "gray20")
```

```
# Scatter plot
p4 <- ggplot(df, aes(x = hp, y = wt)) +
  geom_point(color = "gray20")

# View the plots
p1
p2
p3
p4
```



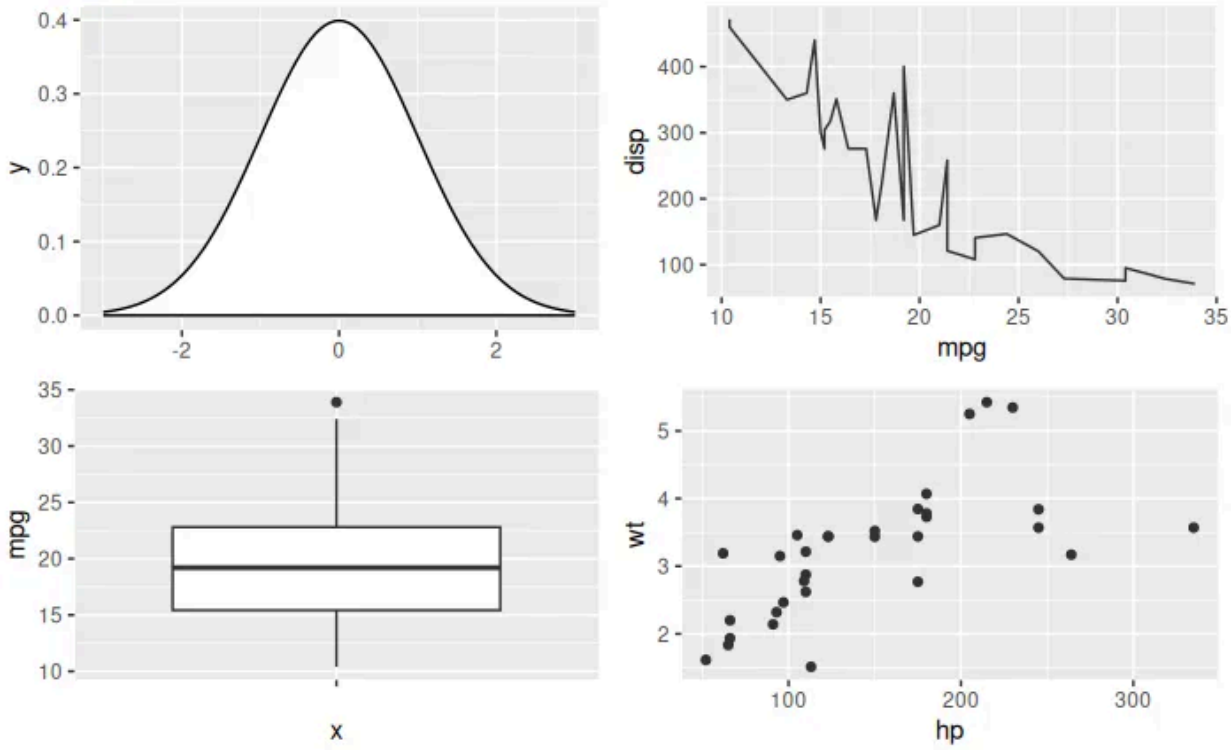



## gridExtra

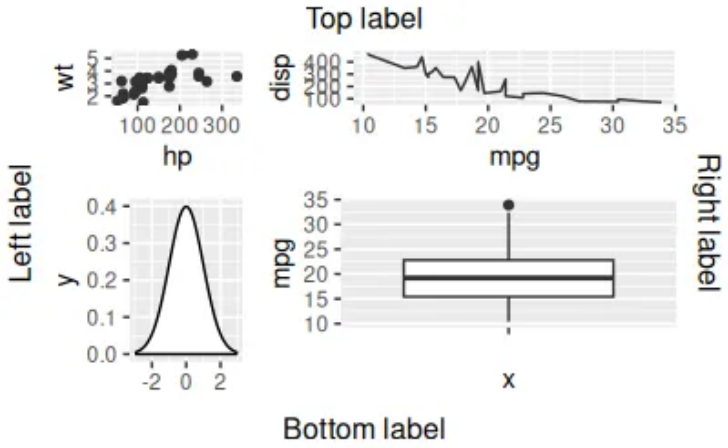
The **gridExtra** package provides the **grid.arrange** function to combine several plots on a single figure.

```
# install.packages("gridExtra")
library(ggplot2)
library(gridExtra)

# Combine the plots with gridExtra
grid.arrange(p2, p3, p1, p4)
```



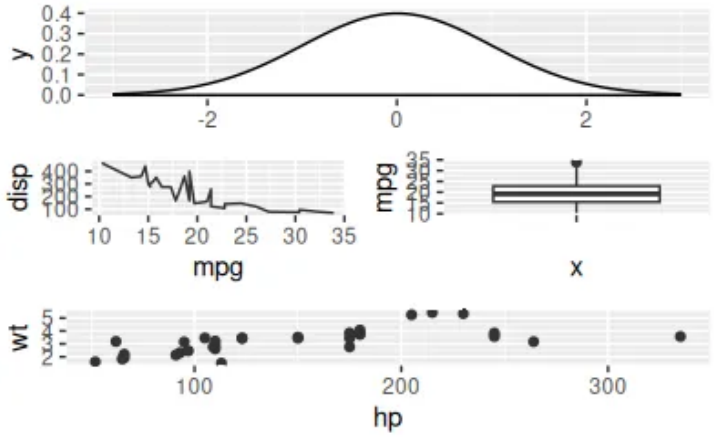

You can also specify the number of rows with **nrow**, the number of columns with **ncol**, and the sizes with **widths** and **heights,** and also we can add labels at the top, bottom, left, and right of the figures.

```
library(ggplot2)
library(gridExtra)

# Combine the plots
grid.arrange(p4, p3, p2, p1,
             top = "Top label", bottom = "Bottom label",
             left = "Left label", right = "Right label",
             widths = c(1, 2), heights = c(2, 3))
```




We can create a layout matrix indicating the positions for each plot and use the **layout_matrix** function in order to specify the desired layout.

```
library(ggplot2)
library(gridExtra)

# layout
layout <- matrix(c(1, 1,
                   2, 3,
                   4, 4), ncol = 2, byrow = TRUE)

grid.arrange(p2, p3, p1, p4,
             layout_matrix = layout)
```

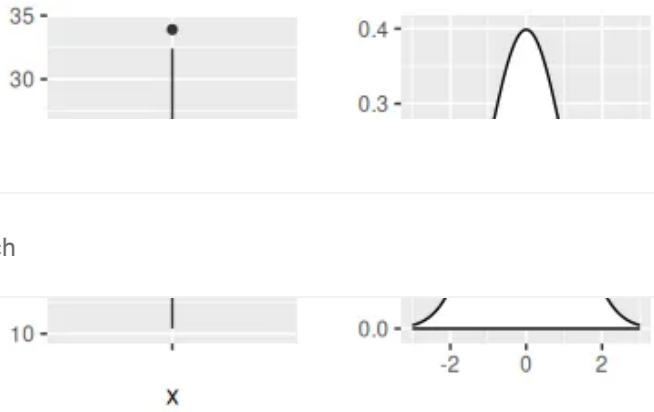



## patchwork

## Combining ggplot2 plots

**patchwork** is designed to combine **ggplot2** plots into the same figure easily by using the + operator to combine the charts.

```
# install.packages("patchwork")
library(ggplot2)
library(patchwork)
# Combine the plots
p1 + p2
```

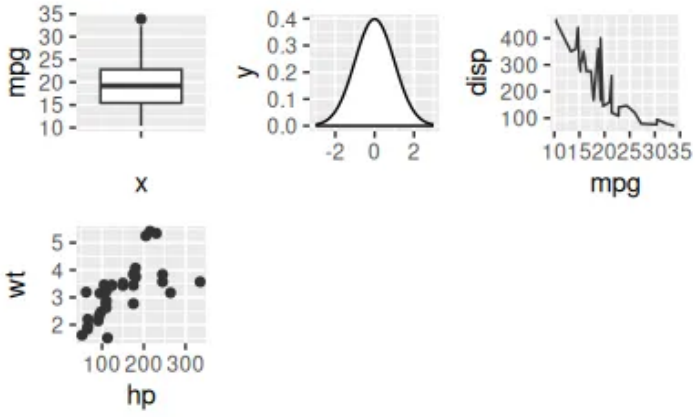





## Controlling the layout

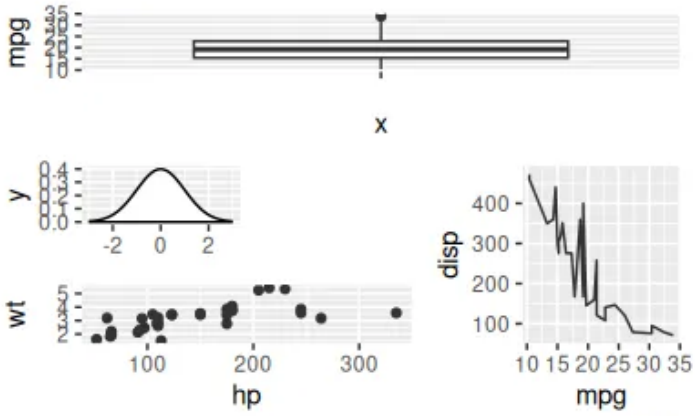
If you want to customize the number of rows or columns of the figure you can use the **plot_layout** function. Also, you can also specify the widths and heights of the plots with widths and heights arguments.

```
library(ggplot2)
library(patchwork)
# Combine the plots
p1 + p2 + p3 + p4 +
 plot_layout(ncol = 3)
```




The most interesting functionality of the plot_layout function is that you can create a custom layout design as shown in the example below, where 1, 2, 3, and 4 represent the locations for p1,p2, p3, and p4, respectively, and # represents an empty space. Recall that you can use numbers but also letters to represent the plot locations.

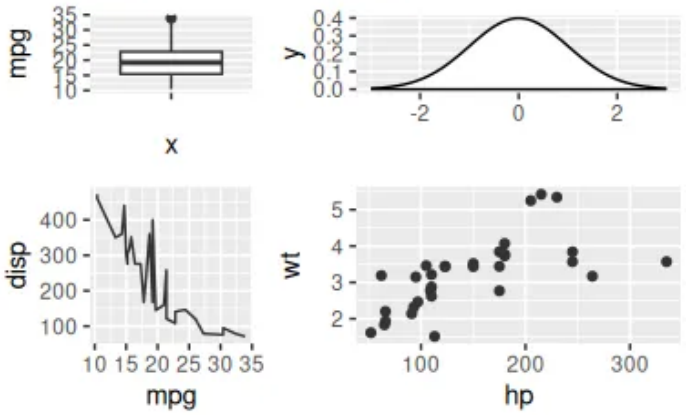
```
library(ggplot2)
library(patchwork)
# Combine the plots with a custom layout
p1 + p2 + p3 + p4 +
 plot_layout(design ="111
 2#3
 443")
```




## The wrap_plots function

Sometimes you can't use the + operator programmatically, so if you don't know the number of plots beforehand you can use the **wrap_plots** function and pass a list of plots to it. This function also allows specifying the number of rows and columns, the sizes, and the custom layouts.

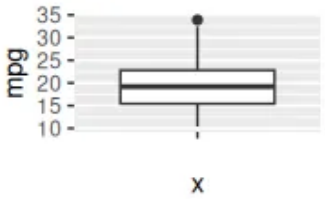
```
library(ggplot2)
library(patchwork)
# Combine the plots
wrap_plots(p1, p2, p3, p4,
 ncol = 2, nrow = 2,
 widths = c(1, 0.5), heights = c(0.5, 1))
```

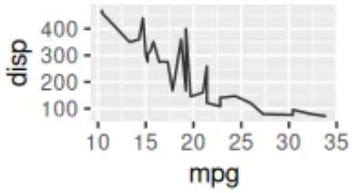



## Adding spacers

When creating a custom layout you can use # to add spaces, as shown in one of the previous examples, but if you are using + there is also a function named **plot_spacer** to add spaces or gaps between plots.

```
library(ggplot2)
library(patchwork)
# Plots with spaces
p1 + plot_spacer() + plot_spacer() + p3
```



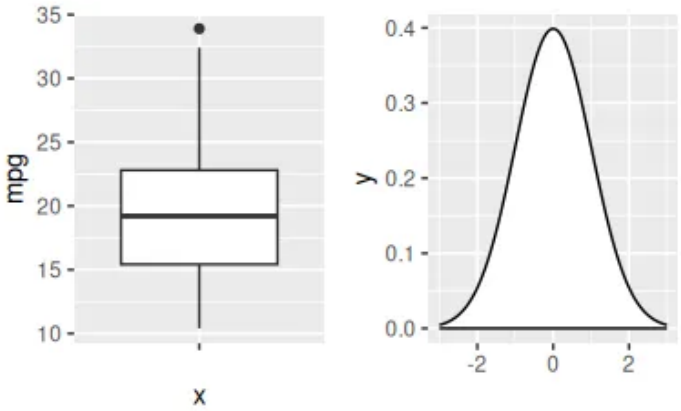

## Operators

The patchwork package also provides two operators to place plots beside each other or to stack them.

### Arranging ggplot2 plots in rows (beside each other)

The | operator places plots in a row. This operator is similar to + when you have two plots but | will place all plots in a single row while + will try to create a square layout if possible.

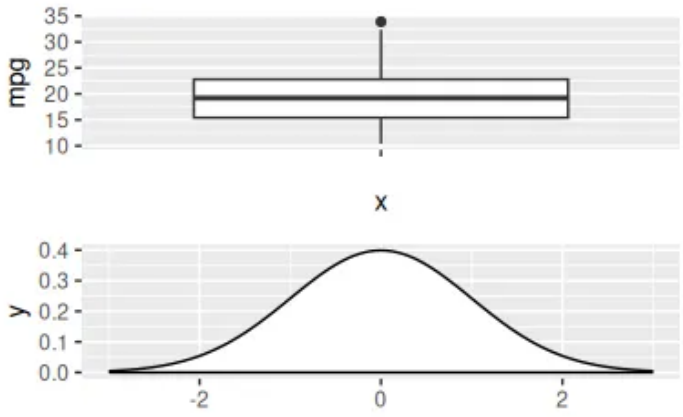
```
library(ggplot2)
library(patchwork)
# Combine the plots in rows
p1 | p2
```



## Arranging ggplot2 plots in columns (stacked)

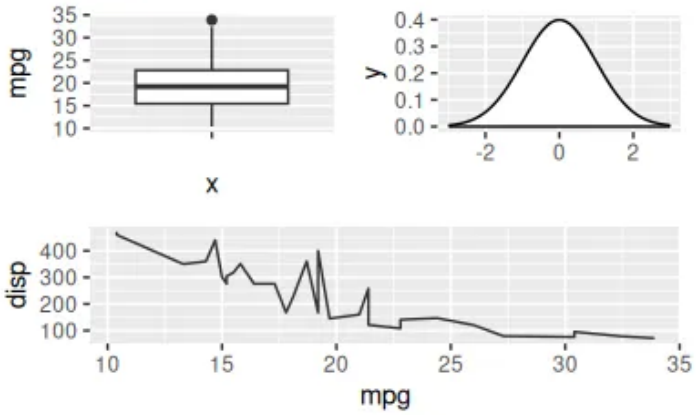
The / operator stacks the ggplot2 plots into columns without the need of using the **plot_layout** function and specifying **ncol = 1**.

```
library(ggplot2)
library(patchwork)
# Combine the plots as column
p1 / p2
```



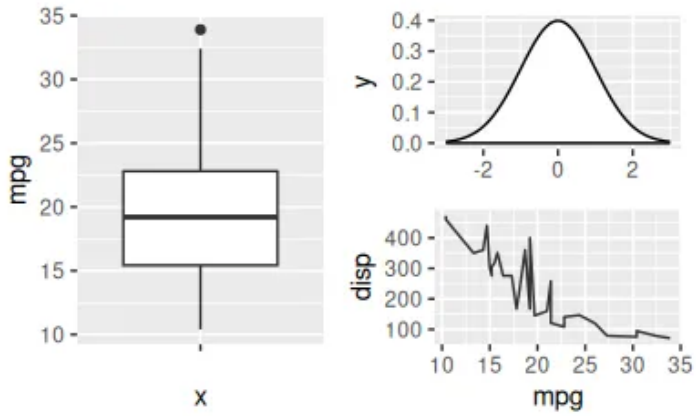

You can create complex layouts. The | and / operators can be used to create complex layouts combining plots. In the following example, we are creating a layout with two plots at the top and one wider at the bottom.

```
library(ggplot2)
library(patchwork)
# Two plots on top and one at the bottom
(p1 | p2) / p3
```




The following example is similar to the previous one, but with one plot on the left and two on the right.

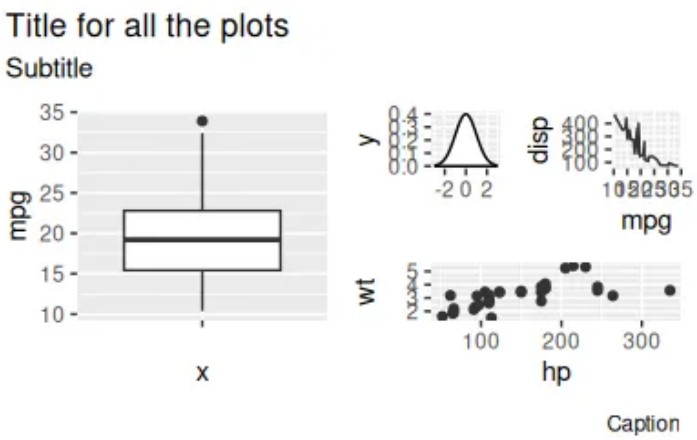
```
library(ggplot2)
library(patchwork)
# One plot at the left and two at the right
p1 | (p2 / p3)
```




## Titles and labels
### Title for all the plots

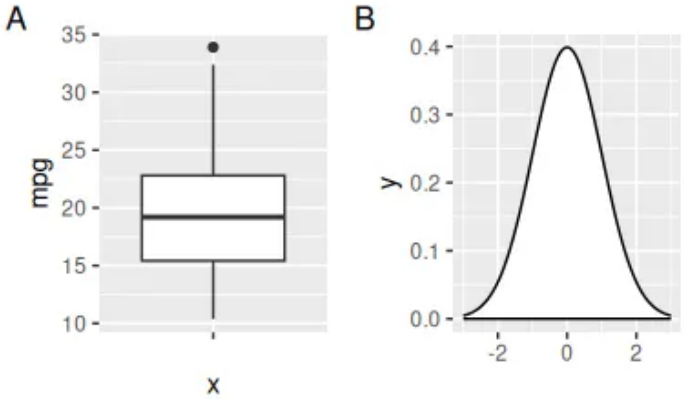
You can add a title, subtitle, and captions to all plots with the **plot_annotation** function.

```
library(ggplot2)
library(patchwork)
# Title for the combined plots
p1 + ((p2 | p3) / p4) +
plot_annotation(title = "Title",
 subtitle = "Subtitle",
 caption = "Caption")
```



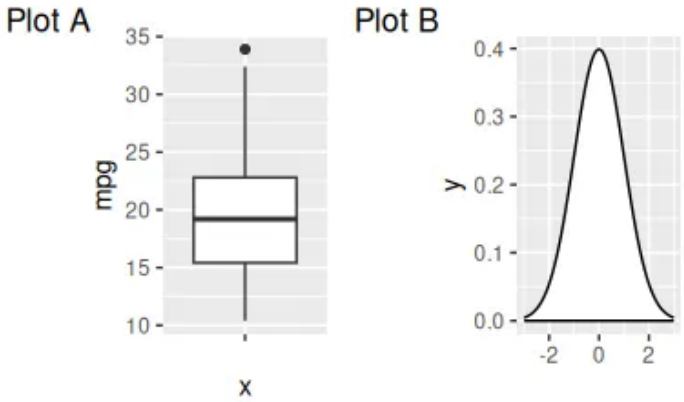

The plot_annotation function can also be used to label each plot individually with the **tags_level** argument. Possible options are "**1**" for numbers, "**a**" for lowercase letters, "**A**" for uppercase letters, "**i**" for lowercase Roman numerals, "**I**" for uppercase Roman numerals, or a vector with your own tags.

```
library(ggplot2)
library(patchwork)
# Labels for each plot
p1 + p2 + plot_annotation(tag_levels = "A")
```




The labels can be customized with the **tag_prefix**, **tag_suffix,** and **tag_sep** arguments.

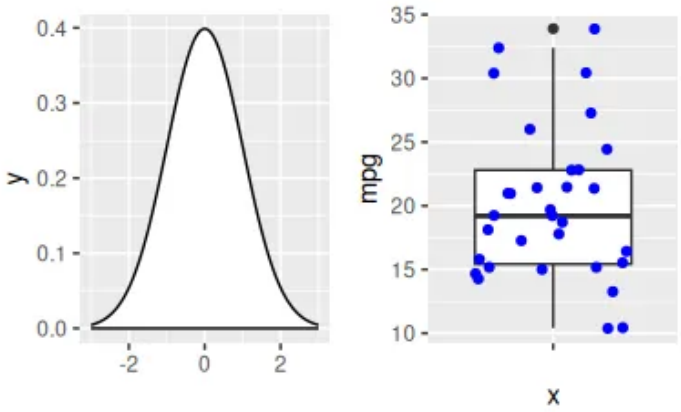
```
library(ggplot2)
library(patchwork)
# Labels for each plot
p1 + p2 + plot_annotation(tag_levels = "A", tag_prefix = "Plot ")
```




### Adding more layers

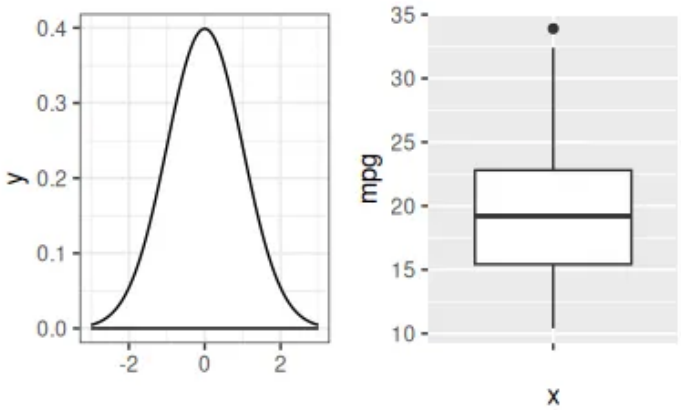
The figures created with patchwork behave the same way as a **ggplot2** object, so you can add new layers as with normal plots, but the layer will be applied to the last added plot.

```
library(ggplot2)
library(patchwork)
# Add a new layer
p2 + p1 +
 geom_jitter(color = "blue")
# Equivalent to:
p <- p2 + p1
p + geom_jitter(color = "blue")
```




If you want to customize other than the last plot added you can add the new layer to it or save the patchwork, access the desired element, and customize it, as shown in the following example.

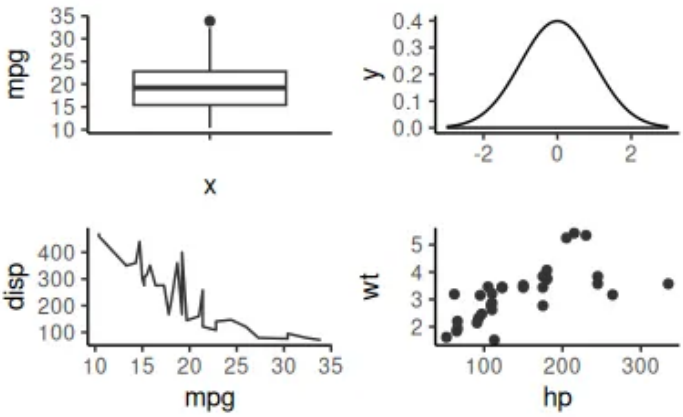
```
library(ggplot2)
library(patchwork)
# Add a new layer to the first plot
p2 + theme_bw() + p1
# Equivalent to:
p <- p2 + p1
p[[1]] <- p[[1]] + theme_bw()
p
```




### Modifying all plots at the same time

**patchwork** also provides the & operator to modify all the plots at the same time to set the same theme for all plots at the same time.

```
library(ggplot2)
library(patchwork)
# Change the theme for all plots
p1 + p2 + p3 + p4 & theme_classic()
```

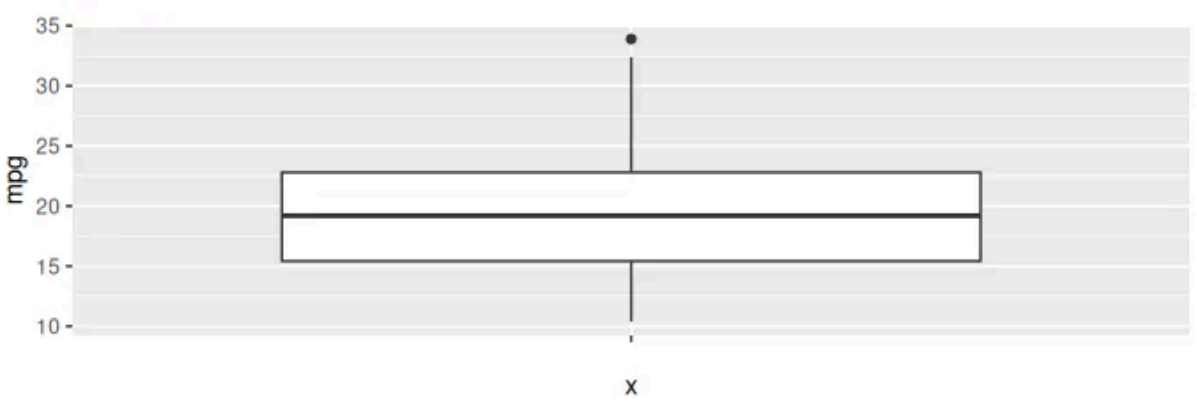



### Adding tables and text

When adding base R plots and ggplot2 plots patchwork won't be able to align the plots, so you will need to customize the margins for one of the plots and try to fine-tune the values until you reach a good alignment.

Making use of the **tableGrob** function from the **gridExtra** package you can add a table to a layout created with patchwork.

```
# install.packages("gridExtra")
library(ggplot2)
library(patchwork)
library(gridExtra)
tab <- t(round(quantile(df$mpg), 2))
# ggplot2 with table
p1 / tableGrob(tab)
```

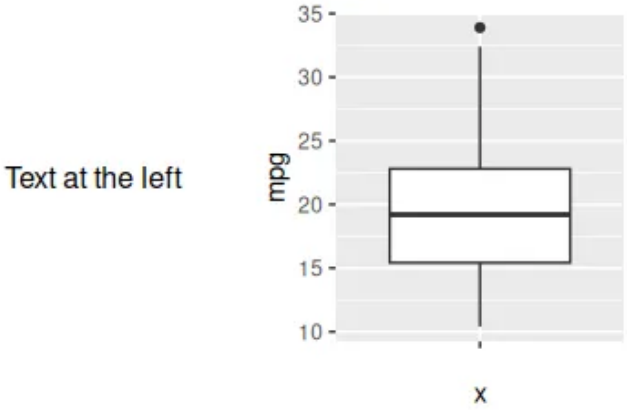



| 0% | 25% | 50% | 75% | 100% |
|---|---|---|---|---|
| 10.4 | 15.43 | 19.2 | 22.8 | 33.9 |

You can also use the **textGrob** function from **gridExtra** to add text to the layout, but note that if you want the text to be the first element you will need to use the **wrap_elements** function.

```
library(ggplot2)
library(patchwork)
library(grid)
```

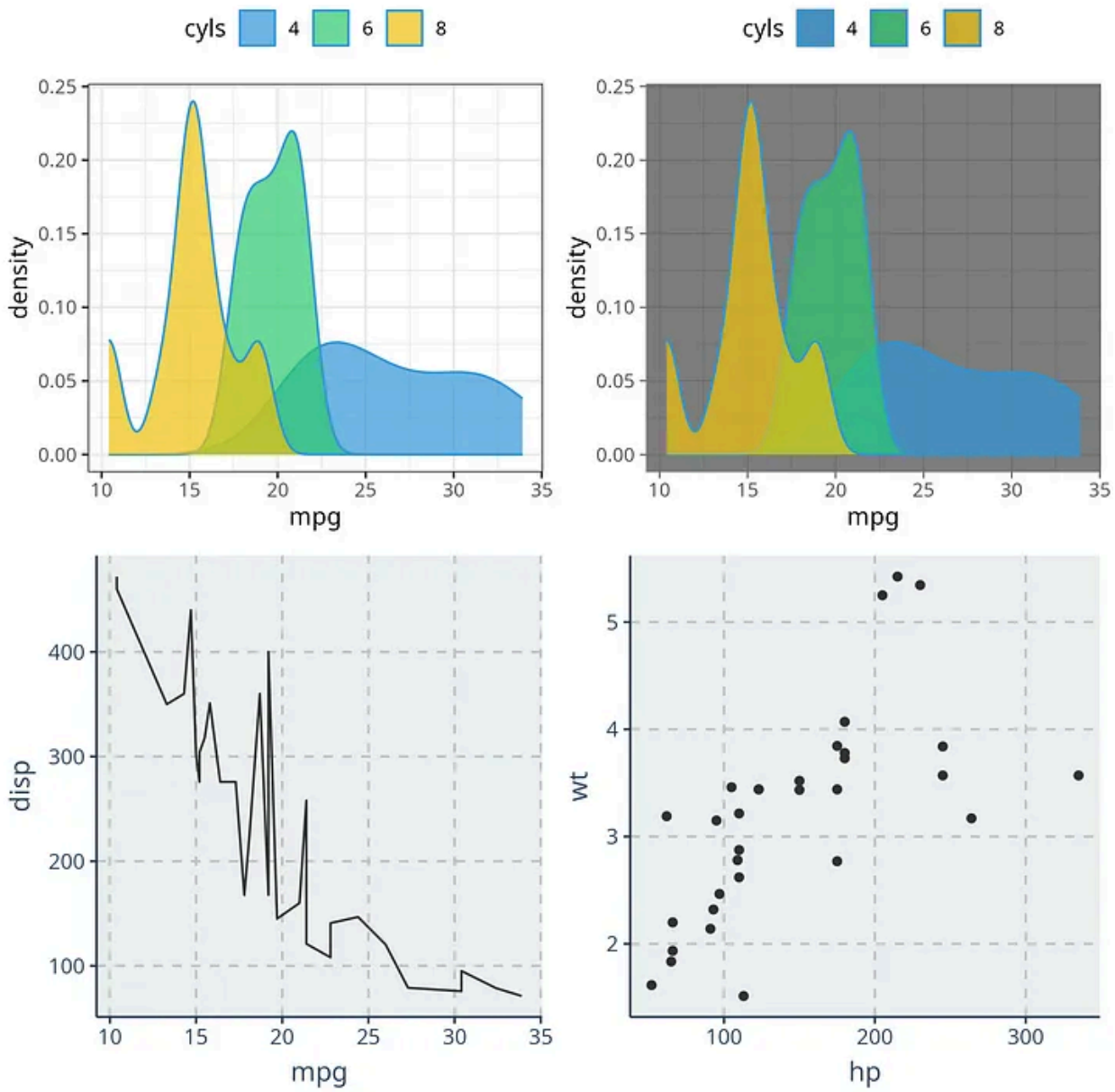
```
# ggplot2 with text
p1 + textGrob("Text at the right")
# To put the text first use:
wrap_elements(textGrob("Text at the left")) + p1
```




## cowplot

The **cowplot** package combined plots using the **plot_grid** function.

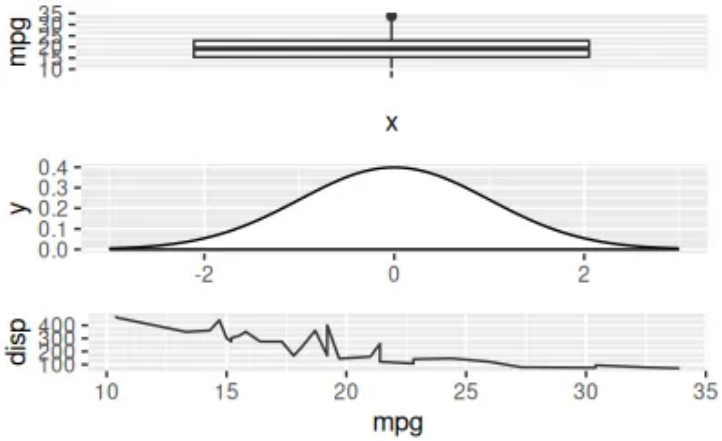
```
# install.packages("cowplot")
library(ggplot2)
library(cowplot)
plot_grid(p1, p2, p3, p4)
```




## Aligning the plots

The **plot_grid** function also customizes the number of rows and columns of plots with the nrow and ncol arguments.

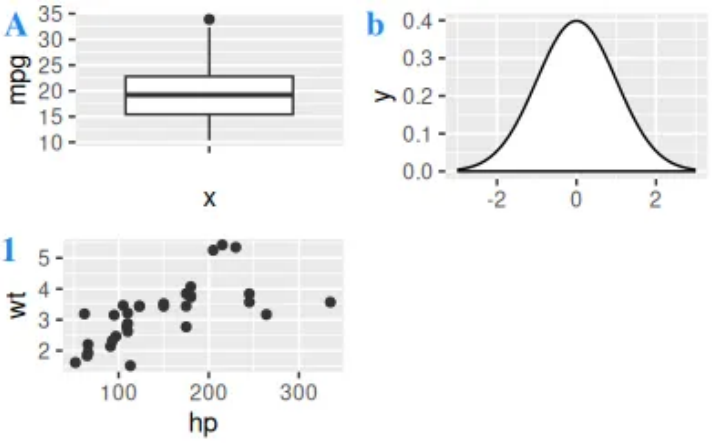
```
library(ggplot2)
library(cowplot)
plot_grid(p1, p2, p3, ncol = 1)
```




## Adding labels to each plot

If you want to label each plot individually you can make use of the labels argument of the function, where you can specify a vector of labels or use the "**A**" or "**a**" keywords for automatic labels in uppercase or lowercase, respectively. The function also provides several arguments to customize the style of the texts.

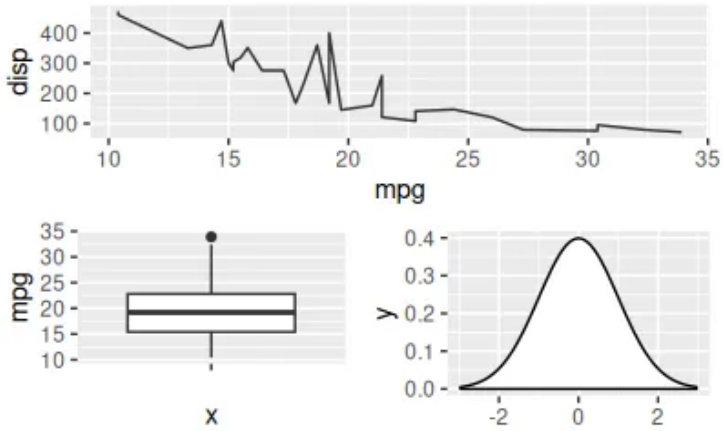
```
library(ggplot2)
library(cowplot)
plot_grid(p1, p2, p4,
 labels = c("A", "b", 1),
 label_fontfamily = "serif",
 label_fontface = "bold",
 label_colour = "dodgerblue2")
```



## Mixing plots with cowplot

With **cowplot** you can also create more complex layouts combining **plot_grid** functions, as shown in the example below, where we are creating a layout with two plots at the bottom and one at the top.

```
library(ggplot2)
library(cowplot)
# Grid layout with cowplot
plot_grid(p3, plot_grid(p1, p2), ncol = 1)
```




Ggplot2 R Data Visualization Data Science

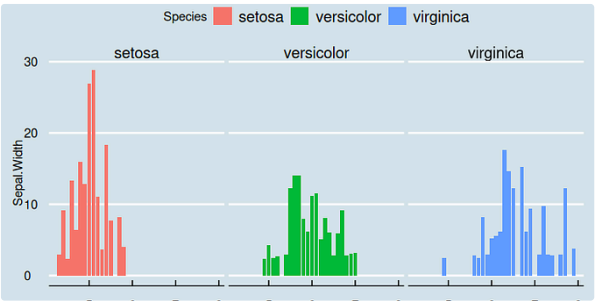
# Written by Pawan

Follow

8 Followers

Reseach Assistant

## More from Pawan

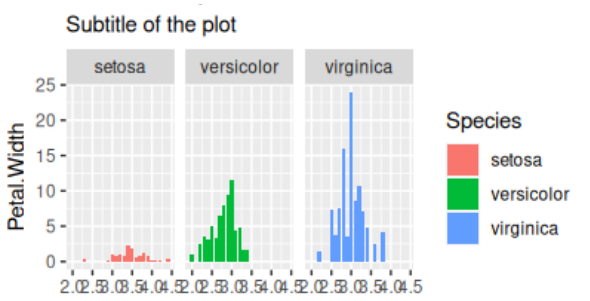



Pawan

### ggplot2: Themes of the plot

In the ggplot2 package, there are two types of themes, one is a theme function that is used...

Jan 7, 2023 12




Pawan

### ggplot plot title, subtitle, caption and tag

When using ggplot2 we can set a title, subtitle, caption, and tag of the plot. There ar...

Sep 14, 2023 2

Pawan

### Data in R

Data types

Sep 16, 2023 1

See all from Pawan

## Recommended from Medium

Pierre DeBois in CodeX

### What's New in The GT Library Version 0.11 That Makes R...

The latest version of the gt library for R programming offers LaTeX styling, chemical...

Aug 7 12

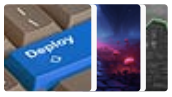
Eliana Ibrahimi in Stats Learning

### Simple Linear Regression in R

Understanding Simple Linear Regression in R: From Concept to Code

Jul 27 143 2

## Lists



**Predictive Modeling w/ Python**

20 stories · 1457 saves

**ChatGPT prompts**

48 stories · 1919 saves

**Coding & Development**

11 stories · 751 saves

**Practical Guides to Machine Learning**

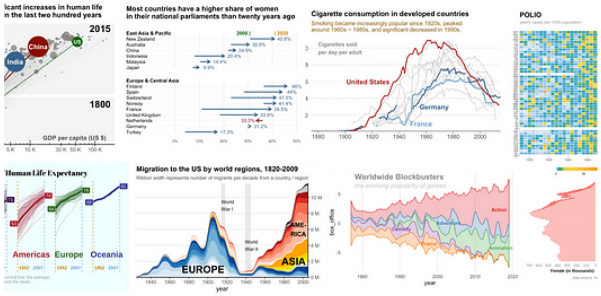
10 stories · 1772 saves

Farzad Mahmoodinobar in Towards Data Science

## Multivariate Analysis—Going Beyond One Variable At A Time

Multivariate analysis and visualization in Python

Jan 12, 2023 111 3




Bo Yuan, Ph.D.

## Awesome Strategies to Visualize Change with Time

May 8 1.4K 14


Marcus Gong, PMP®

## Unlock the Power of Loops in R

Discover how loops in R can streamline your data processes, turning complex tasks into…

Jul 27 7

Kamol C. Roy

## Create Dynamic Geo-Spatial Visualization using Manim

I was looking for video content to better understand neural networks. Then, when I…

Mar 3 6 1

See more recommendations